SYNCRA

Contrato de Dados

Plataforma de criação, distribuição e marketplace de chatbots inteligentes

Campo	Conteúdo
Documento	02 — Contrato de Dados
Versão	0.1 — minuta para revisão da equipe
Data de emissão	Agosto de 2026
Situação	Em revisão. As seções 3 e 6 dependem das decisões 8.2 e 8.4 do documento de arquitetura.
Autores	Vinicius Prado Sobral, Tarso Monteiro Alves Passos, João Guilherme Costa Carvalho, Gabriel Henrique Costa Novaes
Documentos relacionados	00 — Visão e Escopo v1.2 · 01 — Arquitetura v0.1 · 03 — Plano de Execução

> **Por que este documento existe**
>
> Define o formato canônico do bot e o esquema do banco. É o contrato que os três modos de criação obedecem: enquanto todos escreverem neste formato, um bot gerado por prompt abre no editor visual sem perda, que é o critério de aceite CA-02. Alterações aqui afetam código já escrito e exigem migração — por isso este é o documento mais caro de mudar depois.

1. Convenções

- Identificadores: UUID v4 em todas as tabelas, gerados pela aplicação
- Datas: timestamp com fuso horário, sempre em UTC; conversão para horário local é responsabilidade da interface
- Nomes: tabelas no plural e em snake_case; campos em snake_case
- Exclusão: lógica por padrão, com campo de data de exclusão; exclusão física reservada a pedidos de titular de dados
- Valores monetários: inteiros em centavos, nunca ponto flutuante
- Enumerações: armazenadas como texto com restrição de valores, não como inteiro, para manter legibilidade em consultas

2. Entidades

Entidade	Papel	Observação
conta	Cliente da Syncra, pessoa ou organização.	Unidade de cobrança e de isolamento de dados.
usuario	Pessoa que acessa o painel.	Pertence a uma conta.
assinatura	Vínculo entre conta e plano, com estado.	Alimentada por webhooks.

Entidade	Papel	Observação
plano	Configuração comercial: teto de mensagens e limites.	Valores parametrizáveis.
bot	Identidade estável do bot.	Não guarda o fluxo — apenas metadados.
bot_versao	Definição do bot em um instante.	Contém o fluxo. Imutável após publicada.
catalogo_item	Bot da Syncra exposto na vitrine.	Aponta para uma versão específica.
conversa	Sessão entre um usuário final e um bot.	Vinculada à versão com que iniciou.
mensagem	Turno individual da conversa.	Guarda custo e latência da chamada.
consumo	Contagem agregada por bot e período.	Base do teto de mensagens.

> **A separação entre bot e bot_versao**
> O bot é a identidade; a versão é o conteúdo. Sem essa divisão não há como manter uma conversa em andamento presa à definição com que começou, nem como reverter uma edição que quebrou o fluxo. Uma vez publicada, a versão nunca é alterada — editar cria uma nova.

3. Formato Canônico do Bot

Esta é a estrutura que o editor visual grava, que o gerador por prompt precisa produzir e que o motor de execução consome. Um único formato para os três modos de criação.

3.1 Estrutura geral

```
{
  "versao_schema": 1,
  "nome": "Atendimento — dúvidas de matrícula",
  "modelo_padrao": "modelo-rapido",
  "instrucao_global": "Você atende alunos. Seja objetivo.",
  "no_inicial": "no_boas_vindas",
  "variaveis": [
    { "chave": "nome_aluno", "tipo": "texto", "obrigatoria": false }
  ],
  "nos": [ ... ],
  "conexoes": [ ... ],
  "metadados_editor": { "posicoes": { "no_boas_vindas": { "x": 120, "y": 80 } } }
}
```

O campo metadados_editor guarda apenas o que é visual: posição dos nós na tela. O motor de execução ignora esse campo por completo. Isso permite que o modo prompt gere um bot sem posições e o editor calcule um layout automático ao abrir, sem que a ausência quebre nada.

3.2 Tipos de nó

Tipo	Função	Campos próprios
mensagem	Envia texto fixo ao usuário.	texto
pergunta	Envia texto e aguarda resposta, gravando em	texto, variavel_destino

Tipo	Função	Campos próprios
	variável.	
ia	Gera resposta pelo modelo, com instrução específica do nó.	instrucao, modelo, usar_historico
condicao	Ramifica o fluxo conforme uma expressão sobre variáveis.	expressao
encerramento	Finaliza a conversa.	texto_final

Cinco tipos cobrem o escopo da Fase 1. A lista é deliberadamente curta: cada tipo novo precisa ser suportado simultaneamente pelo editor, pelo gerador por prompt e pelo motor, então crescer aqui custa três vezes.

3.3 Nó e conexão

```
// nó
{
  "id": "no_coleta_nome",
  "tipo": "pergunta",
  "texto": "Qual é o seu nome?",
  "variavel_destino": "nome_aluno"
}

// conexão
{
  "de": "no_coleta_nome",
  "para": "no_resposta_ia",
  "condicao": null        // preenchida apenas quando a origem é do tipo condicao
}
```

3.4 Regras de validação

Toda definição passa pelo validador antes de ser gravada, independentemente do modo que a produziu. As regras abaixo são bloqueantes.

#	Regra	Motivo
V-01	no_inicial precisa existir na lista de nós.	Sem ponto de entrada não há execução.
V-02	Todo id de nó é único dentro do bot.	Referência ambígua em conexões.
V-03	Toda conexão aponta para nós existentes.	Evita fluxo interrompido em execução.
V-04	Todo nó é alcançável a partir do nó inicial.	Nó órfão indica erro de edição.
V-05	Todo nó do tipo condicao tem ao menos duas saídas.	Ramificação com saída única não é ramificação.
V-06	Toda variável referenciada foi declarada.	Evita leitura de valor inexistente em tempo de conversa.
V-07	O grafo não contém ciclo sem nó de espera de entrada.	Impede laço infinito consumindo chamadas ao modelo.
V-08	Existe ao menos um caminho até um nó de encerramento.	Conversa sem fim definido nunca libera a sessão.

> **Onde o validador vive**
>
> Um único esquema, escrito em Zod, compartilhado entre frontend e backend. O frontend o usa para avisar o usuário durante a edição; o backend o reaplica antes de gravar. Duplicar essa regra em dois lugares é o caminho mais curto para os dois divergirem.

4. Esquema do Banco

4.1 contas e usuarios

```
contas
  id                uuid        PK
  nome              text        not null
  criado_em         timestamptz not null
  excluido_em       timestamptz null

usuarios
  id                uuid        PK
  conta_id          uuid        FK -> contas.id
  email             text        not null unique
  nome              text        not null
  papel             text        not null   // proprietario | membro
  criado_em         timestamptz not null
```

4.2 planos e assinaturas

```
planos
  id                    uuid        PK
  nome                  text        not null
  preco_centavos        integer     not null
  teto_mensagens_mes    integer     null   // null = sem teto; ver doc 00, 4.4
  max_bots              integer     not null
  ativo                 boolean     not null

assinaturas
  id                    uuid        PK
  conta_id              uuid        FK -> contas.id
  plano_id              uuid        FK -> planos.id
  estado                text        not null   // ativa | inadimplente | cancelada |
teste
  referencia_externa    text        null       // id no provedor de pagamento
  inicio_em             timestamptz not null
  fim_em                timestamptz null

eventos_pagamento
  id                    uuid        PK
  assinatura_id         uuid        FK -> assinaturas.id
  tipo                  text        not null
  payload               jsonb       not null
  id_evento_externo     text        not null unique   // idempotência
  recebido_em           timestamptz not null
```

O campo id_evento_externo com restrição de unicidade é o que impede que um webhook reenviado pelo provedor seja processado duas vezes. Provedores reenviam por padrão quando não recebem confirmação, então isso não é caso raro.

4.3 bots e versões

```
bots
  id                    uuid        PK
  conta_id              uuid        FK -> contas.id
  nome                  text        not null
  versao_publicada_id   uuid        FK -> bot_versoes.id  null
  origem_catalogo_id    uuid        FK -> catalogo_itens.id null  // informativo
  ativo                 boolean     not null
  criado_em             timestamptz not null
  excluido_em           timestamptz null

bot_versoes
  id                    uuid        PK
  bot_id                uuid        FK -> bots.id
  numero                integer     not null   // sequencial por bot
  definicao             jsonb       not null   // formato canônico da seção 3
  modo_criacao          text        not null   // visual | prompt | consultora
  publicada_em          timestamptz null
  criado_em             timestamptz not null
  unique (bot_id, numero)
```

O campo modo_criacao não tem função na execução. Existe para medir: ao fim do piloto, permite saber qual modo os usuários realmente usaram, o que informa onde investir esforço na fase seguinte.

4.4 catálogo

```
catalogo_itens
  id                    uuid        PK
  bot_versao_id         uuid        FK -> bot_versoes.id
  titulo                text        not null
  descricao             text        not null
  categoria             text        not null
  preco_centavos        integer     not null   // 0 = gratuito
  publicado             boolean     not null
  publicado_em          timestamptz null

aquisicoes
  id                    uuid        PK
  conta_id              uuid        FK -> contas.id
  catalogo_item_id      uuid        FK -> catalogo_itens.id
  bot_gerado_id         uuid        FK -> bots.id
  adquirido_em          timestamptz not null
```

A aquisição copia a definição para uma bot_versao nova, pertencente à conta do cliente. O vínculo com o item de origem fica registrado apenas para histórico: alterações posteriores no item do catálogo não alcançam as cópias já distribuídas.

4.5 conversas e mensagens

```
conversas
  id                    uuid        PK
  bot_id                uuid        FK -> bots.id
  bot_versao_id         uuid        FK -> bot_versoes.id   // versão travada no início
  canal                 text        not null   // web | whatsapp (ver decisão 8.2)
  identificador_externo text        null       // id do usuário final no canal
  no_atual              text        null
  variaveis             jsonb       not null default '{}'
  estado                text        not null   // ativa | encerrada | bloqueada
  consentimento_em      timestamptz null       // LGPD
```

```
  iniciada_em           timestamptz not null
  encerrada_em          timestamptz null
  expurgo_em            timestamptz null       // data-limite de retenção

mensagens
  id                    uuid        PK
  conversa_id           uuid        FK -> conversas.id
  papel                 text        not null   // usuario | bot | sistema
  conteudo              text        not null
  no_id                 text        null       // nó que originou a mensagem
  tokens_entrada        integer     null
  tokens_saida          integer     null
  custo_inferencia_centavos integer null
  custo_canal_centavos  integer     null
  latencia_ms           integer     null
  criada_em             timestamptz not null
```

> **Dois custos, dois campos**
> Custo de inferência e custo de canal são separados desde a primeira migração. Somados em um único campo, torna-se impossível saber, ao fim do piloto, se a margem está sendo comprimida pelo modelo ou pela cobrança por conversa da Meta — e as duas variáveis exigem respostas opostas.

4.6 consumo

```
consumos
  id                    uuid        PK
  conta_id              uuid        FK -> contas.id
  bot_id                uuid        FK -> bots.id
  periodo               date        not null   // primeiro dia do mês de referência
  mensagens             integer     not null default 0
  custo_total_centavos  integer     not null default 0
  unique (bot_id, periodo)
```

A contagem corrente vive no Redis durante o mês e é consolidada aqui. O Postgres é a fonte de verdade; o Redis é o contador rápido consultado antes de cada chamada ao modelo.

5. Índices e Integridade

- conversas: índice composto em (bot_id, estado) — consulta mais frequente do painel
- mensagens: índice em (conversa_id, criada_em) — montagem da janela de contexto
- bot_versoes: índice em (bot_id, numero desc) — recuperação da versão mais recente
- Toda consulta de leitura filtra por conta_id, sem exceção; o isolamento entre contas é regra de esquema, não de aplicação
- Exclusão de conta não apaga conversas em cascata: o expurgo é processo assíncrono com prazo, para não perder registro de custo

6. Retenção e LGPD

Dado	Retenção	Base
Conteúdo de mensagens	Prazo definido por plano, com padrão a	Campo expurgo_em na conversa.

Dado	Retenção	Base
	confirmar.	
Metadados de custo e latência	Mantidos após o expurgo do conteúdo.	Registro financeiro sem dado pessoal.
Identificador do usuário final	Removido junto com o conteúdo.	Anonimização da conversa.
Consentimento	Mantido enquanto durar a relação.	Campo consentimento_em.

O expurgo apaga conteúdo e identificadores, preservando a linha da conversa com os campos de custo. Assim a conta continua auditável financeiramente depois que os dados pessoais deixaram de existir.

7. Pontos em Aberto

#	Ponto	Depende de
7.1	Campo canal em conversas: manter, ou remover se a Fase 1 for canal único.	Decisão 8.2 do documento de arquitetura.
7.2	Formato de armazenamento do estado da conversa: jsonb livre ou colunas tipadas.	Decisão 8.4 — interpretação do grafo ou máquina de estados.
7.3	Prazo padrão de retenção de conversas.	Documento de conformidade LGPD.
7.4	Valor de teto_mensagens_mes nos planos.	Modelo de cobrança da Meta (doc 00, seção 4.4).
7.5	Necessidade de uma tabela de execuções para depuração de fluxo.	Avaliação após os primeiros bots reais.

8. Controle de Versão

Versão	Data	Descrição	Responsável
0.1	Ago/2026	Minuta inicial para revisão da equipe	Equipe Syncra

Alterações no formato canônico após o início do desenvolvimento exigem incremento de versao_schema e rotina de migração das definições já gravadas.